Northeastern University, Khoury College of Computer Science

# CS 6120 Natural Language Processing Project Report

## What do we do with all these food in fridge?

**Xiaoyao Xu**
xu.x1@northeastern.edu
GitHub: XrayBravoGolf/RecipesRAG

## 1 Purpose

This project aims to build an experimental, verifiable Retrieval-Augmented Generation (RAG) culinary assistant that helps users utilize their available ingredients by recommending recipes and providing traceable source evidence.

## 2 Background

Retrieval-Augmented Generation (RAG) reduces Large Language Model (LLM) hallucinations by providing external context, but trusting the output remains challenging when specific constraints matter. "Verifiable RAG" solves this by tightly coupling generated text with exact source citations, allowing users to trace claims. In the culinary domain, combining vector similarity search with a generative assistant offers a transparent way to suggest recipes and ingredient substitutions while citing exact sources.

## 3 Approach & Implementation

### 3.1 Data Processing & Storage

This project utilizes the corbt/all-recipes dataset from HuggingFace, which consists of 2,147,248 examples in its training split, stored in Parquet format with a single text string per recipe. Because each row contains a complete set of ingredients and instructions, we are able to skip traditional chunking algorithms or sliding window overlaps, using the per-recipe separation as our document unit.

For similarity search, we use a local FAISS vector database configured with IndexFlatIP and wrapped in IndexIDMap2 to map embeddings to global IDs. To maintain verifiability without significantly increasing the size of the vector index, the raw recipe_text and exact file coordinates (parquet_filename, row_in_file, and global_row_id) are stored in an accompanying SQLite relational database (metadata.sqlite).

## 3.2 Embedding & Retrieval

### Embedding

We use the BAAI/bge-small-en-v1.5 pre-trained embedding model without any additional training or fine-tuning. It was chosen for its performance-to-size ratio on retrieval benchmarks and its 384-dimensional output, which helps keep the 2-million-row vector index manageable in memory. To handle the scale of the dataset during index generation, the system streams records directly from Parquet block groups for batch processing using an insert batch size of 1024 and an encode layout size of 512. The embedding build process was executed on an NVIDIA RTX 3080 Ti GPU, fully utilizing its 12GB of VRAM to complete the operation in approximately 40 minutes. To maintain verifiable source tracking, each FAISS vector insertion is simultaneously paired with an SQLite record containing the corresponding global_row_id, parquet_filename, row_in_file, and recipe_text.

### VERIFIABLE Retrieval through Metadata Database

The retrieval pipeline executes a dense vector similarity search using inner product distances within the FAISS index to find the most relevant embeddings. Because FAISS only stores numerical vectors and their corresponding IDs efficiently, we rely on a separate local SQLite metadata database to maintain verifiable evidence and human-readable context. When a user sends a query, FAISS quickly returns the global_row_id and distance score for the top-k matches. These IDs are passed to the SQLite database to fetch the recipe_text and source tracking coordinates (parquet_filename and row_in_file). This two-step retrieval architecture reduces the vector index size while providing verifiable source tracing.

## 3.3 Generation Engine & Remediation

The generation step uses the Qwen/Qwen2.5-3B-Instruct language model. To run this locally on a single GPU node and avoid the overhead of a distributed computation cluster, the model weights are loaded in half-precision bfloat16. Before generation, the retrieved recipes from the FAISS step are aggregated and formatted into a structured context block. The model then processes this context alongside the user's query using its conversational chat template.
A common challenge in RAG systems is preventing the model from relying on its pre-trained internal knowledge when the retrieved context is insufficient. To address this, the system prompt establishes rules for grounding the response:

> *"You are a helpful culinary assistant. Use ONLY the provided recipes to answer the user's question. If a recipe contains the answer, show the recipe in a way that both answers user's question and illustrates the cooking. If the recipes do not contain the answer, say 'I cannot answer this based on the provided recipes.' However, in this case include one recipe in your answer and indicate it is the closest match based on the question. Do not use outside knowledge. However, if the user asks for substitutions or modifications, provide reasonable suggestions based on the ingredients in the recipes."*

This prompt provides guidelines for utilizing the provided recipes versus suggesting ingredient substitutions. To support this behavior, the model's generation temperature is set to 0.3 with sampling enabled. This configuration restricts output variance. The system provides the model with up to 5 retrieved recipes by default (Top-k = 5) to balance context size against noise. Finally, the maximum sequence length for new tokens is set to 1400 to provide sufficient tokens to output instructions and make sure a user finishes their culinary answer.

### 3.4 Serving & UI

The application is served using a Streamlit frontend (frontend.py), which is used because it is easy to configure and iterate on. Running directly on the Northeastern HPC cluster through interactive GPU sessions, we used a monolithic script architecture, rather than containerization or orchestration layers. To present the verifiable evidence to the user, the interface features an expander that displays the raw source text, retrieval scores, and database coordinates (global IDs) for each recipe. Users can also download this retrieved evidence as a CSV file to verify the LLM's references. During the evaluation session, one NVIDIA V100 GPU was allocated to run the application, handling both the retrieval and generation workloads.

### 3.5 Validation & Evaluation

Results are validated primarily through manual inspection by the author and classroom peers, where the LLM's generated culinary suggestions are compared against the source recipes displayed in the user interface. For a more structured evaluation, the system allows for cross-referencing through the downloadable CSV evidence exports. This is used to check if the language model synthesized the retrieved instructions and respected ingredient constraints without inventing ungrounded steps or non-existent ingredients.

Settings

Top-K

3

Show retrieved context

Clear chat history

# Let's clear your fridge!

Verifiable RAG running on Streamlit frontend

Ask a question to begin. (Note: I cannot answer follow-up questions.)

make pizza without tomato-based sauce

Based on the recipes provided, Recipe 3 seems to be the closest match for making a pizza without a tomato-based sauce. This recipe uses various ingredients like pesto, garlic, and cheese to create a flavorful topping that doesn't rely on tomato sauce. Here's how you can prepare it:

## Ingredients:

- 2 tablespoons melted butter
- 1 tablespoon olive oil
- 3 tablespoons minced garlic
- 2 tablespoons sun-dried tomato pesto
- 1 teaspoon dried basil leaves
- 1 teaspoon dried oregano
- 1 tablespoon grated Parmesan cheese
- 1 unbaked pizza crust
- 1 tomato, sliced
- 1 bunch fresh spinach, torn
- 1 sweet onion, sliced
- 1 fresh jalapeno pepper, chopped
- 1 (6 ounce) package feta cheese, crumbled

## Directions:

1. **Preheat Oven**: Follow the instructions on your pizza crust package to preheat the oven.
2. **Prepare Topping**:
   - In a small bowl, mix the melted butter, olive oil, minced garlic, sun-dried tomato pesto, dried basil, dried oregano, and grated Parmesan cheese until well combined.
   - Spread this mixture evenly over the unbaked pizza crust.
3. **Arrange Vegetables and Cheese**:
   - Place the sliced tomato, torn spinach, sliced onion, and chopped jalapeno pepper on top of the cheese mixture.
   - Sprinkle the crumbled feta cheese over the vegetables.
4. **Bake**:
   - Place the assembled pizza in the preheated oven and bake according to the instructions on your pizza crust package.

This method allows you to create a delicious pizza without using tomato sauce, utilizing the other ingredients to provide flavor and texture.

Retrieved context (Verifiable Evidence)

**Result 1 (Score: 0.8322)**

File: `train-00002-of-00004-3b4f78b99eedadc2.parquet` | Row: `117419` (Global: 1191043)

Recipe 1

Nick Stellino'S No-Cook Pizza Sauce

Ingredients:
- 1 cup tomato sauce
- 1/2 cup tomato paste

**Result 2 (Score: 0.8185)**

File: `train-00001-of-00004-d46654ac93566129.parquet` | Row: `389157` (Global: 925969)

Recipe 2

Easy No-Cook Pizza Sauce

Ingredients:
- 1 (8 ounce) can tomato sauce
- 1 tablespoon onion powder

**Result 3 (Score: 0.8102)**

File: `train-00001-of-00004-d46654ac93566129.parquet` | Row: `412961` (Global: 949773)

Recipe 3

Pizza Without The Red Sauce

Ingredients:
- 2 tablespoons butter, melted
- 1 tablespoon olive oil

Download Evidence as CSV

I want to make pizza without tomato-based sauce